

Unit 3 Part 2

Functions

A function in Python is defined using the `def` keyword, followed by the function name and parentheses that may include parameters. Functions are called using their name followed by parentheses.

```
In [1]: def greet():  
        print("Hello, World!")  
  
greet()
```

Hello, World!

This code defines a function `greet` that prints "Hello, World!" when called. The call `greet()` executes the function.

The Return Statement and Void Function

The `return` statement is used to exit a function and go back to the place where it was called. Functions that don't have a `return` statement or do not return a value explicitly return `None`, which is similar to 'void' functions in other languages.

```
In [2]: def sum(a, b):  
        return a + b  
  
result = sum(5, 3)  
print(result)
```

8

The function `sum` returns the sum of `a` and `b`. The result, 8, is printed.

Parameters and Arguments in a Function

Functions can have three types of parameters: positional, keyword, and default parameters.

Positional Arguments:

Arguments that can be called by their position in the function definition.

Keyword Arguments:

Arguments called by their name.

Default Parameters:

```
In [3]: def describe_pet(animal_type, pet_name='Rex'):
        print(f"I have a {animal_type} named {pet_name}.")

describe_pet('dog')
describe_pet(pet_name='Whiskers', animal_type='cat')
```

```
I have a dog named Rex.
I have a cat named Whiskers.
```

The function is first called with the default pet name, and the second time with keyword arguments in reverse order.

Scope and Lifetime of Variables

Variables declared inside a function are local to that function and cease to exist once the function exits. Variables defined outside are known as global variables and their scope is throughout the module.

```
In [4]: def test_function():
        local_var = 10
        print(local_var)

test_function()
# print(local_var) # This would raise an error because local_var is not de
```

```
10
```

The Return Statement

The return statement sends a function's result back to the caller and terminates the function.

Returning Multiple Values

Functions in Python can return multiple values as tuples.

```
In [5]: def get_person():
        name = "John"
        age = 25
        return name, age # This is a tuple.

person_name, person_age = get_person()
print(person_name, person_age)
```

```
John 25
```

The function returns two values, which are unpacked into `person_name` and `person_age`.

Recursive Functions

Recursion is a fundamental programming concept and technique that involves a function calling itself directly or indirectly to solve a problem. It is a common method for simplifying a complex problem by breaking it down into simpler sub-problems of the same type.

Recursion is often used in solving problems related to data structures and algorithms, such as sorting, searching, and tree and graph traversals.

Key Properties of Recursion

Base Case:

Every recursive function must have a base case, which is the condition under which the function stops calling itself. The base case acts as the termination condition for the recursion, preventing it from continuing indefinitely. Example: In a recursive function that computes the factorial of a number, the base case is when the number is 1, because the factorial of 1 is trivially 1.

Recursive Case:

This is the part of the function that includes the call to itself. The recursive case must alter the input parameters in such a way that they converge towards the base case with each iteration. It typically involves the function calling itself with progressively smaller portions of the input data.

Reduction of Problem Size:

In each recursive call, the problem size should decrease, moving the state of the problem closer to a base case. Failure to reduce the problem size can result in infinite recursion, leading to a stack overflow error.

Maintaining State:

In recursive processes, state is maintained through the function call stack, where each recursive call adds a layer to the stack containing its own local variables and arguments. When a base case is reached, these calls start to return, unwinding the stack back up to the first call.

Multiple Recursive Calls:

Some recursive functions involve more than one recursive call within the recursive case. This is common in problems that branch out, such as tree traversals or algorithms like the QuickSort.

```
In [6]: def factorial(n):  
        # Base case: the factorial of 1 is 1  
        if n == 1:  
            return 1  
        # Recursive case: n * factorial of (n-1)  
        else:  
            return n * factorial(n-1)  
  
print(factorial(5))
```

120

The function is called with $n=5$. It recursively calls itself, decreasing n by 1 each time, until it reaches the base case where $n=1$. As each call reaches the base case, the calls return, multiplying the returned values together ($5 * 4 * 3 * 2 * 1$), resulting in 120.

Advantages and Disadvantages of Recursion

Advantages:

Simplifies complex problems and makes the code easier to write and understand, especially for tasks like tree traversal. Recursive functions can turn complex algorithms into simple, elegant solutions.

Disadvantages:

Can be less efficient and more resource-intensive than iterative solutions due to overhead of maintaining the call stack. Deep recursion can lead to stack overflow errors, especially if the recursion depth is very high or the base case is not reached. It's often harder to debug recursive solutions due to the stack involved.

Lambda Functions

Lambda functions, also known as anonymous functions, are a feature of Python and many other programming languages that allow the creation of small, one-off functions without needing the formal `def` statement. They are called "lambda" functions due to their origin in lambda calculus, which is a formal system in mathematical logic and computer science for expressing computation based on function abstraction and application.

Key Properties of Lambda Functions

Anonymous:

Lambda functions are anonymous, meaning that they are defined without a name. This property makes lambda functions ideal for quick, throwaway functions that are not needed beyond their immediate use.

Syntax:

The syntax of a lambda function in Python is: lambda arguments: expression
Lambda functions can accept any number of arguments, but they can only contain one expression.
The result of this expression is what the function returns when called.

Single Expression:

Unlike regular functions defined with def, which can contain multiple expressions and statements, a lambda function's body is a single expression without any statements. This expression is evaluated and returned when the function is called.

Immediate Use:

Lambda functions are often used in places where a simple function is required for a short period. Common use cases include passing a simple function as an argument to higher-order functions, such as those that require a function for comparison, like sorted(), or for

```
In [7]: words = ['apple', 'banana', 'cherry', 'date']  
sorted_words = sorted(words, key=lambda word: len(word))  
print(sorted_words)
```

```
['date', 'apple', 'banana', 'cherry']
```

The sorted function sorts the list of words based on the length of each word, determined by the lambda function lambda word: len(word). This outputs the list ['date', 'apple', 'banana', 'cherry'], sorted by increasing length of the word.

Using Lambda with map():

```
In [8]: numbers = [1, 2, 3, 4, 5]  
squared_numbers = list(map(lambda x: x**2, numbers))  
print(squared_numbers)
```

```
[1, 4, 9, 16, 25]
```

The map function applies the lambda function to each item of the list numbers. The lambda function squares each number, resulting in the list [1, 4, 9, 16, 25].

Advantages and Disadvantages of Lambda Functions

Advantages:

Conciseness: Lambda functions reduce the amount of code written for small functions.

Flexibility: They can be used inline and are highly useful as arguments to higher-order

functions. No Variables: They typically involve fewer variables and are used where they are created.

Disadvantages:

Limited Functionality: They are limited to a single expression and cannot contain complex logic which includes multiple statements. Readability: For those unfamiliar with lambda functions or when used excessively, they can make the code less readable. Debugging Difficulty: Debugging lambda functions can be more challenging due to their anonymous